

Sprint Plan – Sprint 1 Plan – VeriFi – Completion: June 30

Goal:

Build the skeleton of our front-end where the user will be submitting their inquiries, we will figure out an appropriate design based on references online. We will also have a base level working parser that will generate a .jsonl file.

Story Point Scale (Fibonacci)

Points	Meaning
1	Trivial — under an hour, no uncertainty
2	Small — straightforward, well-understood
3	Medium — some complexity or minor unknowns
5	Large — meaningful effort or real uncertainty
8	Very large — significant complexity or dependency risk
13	Extra large — major unknowns; should be broken down if possible

Infrastructure (3):

- Set up Github Repository
- Write overview README
- Verify Team Access

Spikes (12):

- **Spike 1 (Data Ingestion) (3):** researching and evaluating efficient pdf extraction, chunking, and embedding frameworks and libraries to use
- **Spike 3 (Vector Database) (3):** Figuring out the mathematics to find K-Best chunks of data. Figuring out how to further optimize the algorithm to support $O(N)$ time.
- **Spike 4 (API/Backend) (3):** Researching and learning REST APIs, FastAPI server frameworks, JSON serialization (nlohmann/json), and external Embedding/LLM API integration.
- **Spike 5 (Prompting + QA) (3):** Research Retrieval-Augmented Generation (RAG), prompt engineering techniques, and methods for evaluating AI-generated responses. Design an

initial prompt template and establish quality guidelines for future testing.

Task Listing (Priority Ordered):

User story 1.1

[As a Fidelity user, I want to be able to type a query into a web chatbox and receive a generic text acknowledgment, so that I know the system is successfully receiving my input.]

- **Task 1:** Look up references on the Fidelity website for any uses of chatbots, create a mimic example of the User Interface (3)
- **Task 2:** Code a prototype of the front page the user will access and have a functioning submission text-box that is received by a small mock backend (3)

Total for User Story 1.1: 7 Story points

User story 1.3

[As a system administrator, I want to be able to upload raw financial policy text files into the system, so that the custom semantic vector database is populated with the necessary knowledge to process user queries.]

- **Task 1:** Create a functional small parser that can take in .txt/.pdf files in order to test out parsing capabilities and assess how we want to pass in the chunked information to the backend (3)
- **Task 2:** Create test functions to ensure information is being received on both ends (1)
- **Task 3:** Build a small database where the parsed information is stored, initially without cosine-similarity (1)

Total for User Story 1.3: 5 Story Points

User story 1.2

[As a Fidelity user, I want the chatbox to provide a clickable link to the source document alongside my answer, so that I can verify the information myself.]

- **Task 1:** Figure out a way to extract the links to the documents and structure them within the .jsonl file for referencing within the backend later on in the pipeline (3)
- **Task 2:** Research how retrieved document chunks, metadata, and citations should be formatted before being passed to the LLM for response generation. (3)

Total for User Story 1.2: 6 Story Points

User story 1.4

[As a front-end developer, I want a running /chat endpoint with a defined request/response contract, so that I can begin building the UI against mocked data without waiting on the full pipeline.]

- **Task 1:** Set up a Fast API project w/ venv and dependency management, and verify team machine compatibility (3)
- **Task 2:** Implement GET /health returning {"status": "ok"} with HTTP 200 (1)
- **Task 3:** Implement POST /chat accepting {"query": string} and returning a hardcoded mock response: {"answer": string, "sources": [{doc, page, snippet, score}]} (1)
- **Task 4:** Configure CORS middleware to allow the React frontend on localhost:3000 to communicate with the API (2)
- **Task 5:** Write and share the API contract documentation (request schema, response schema, error format, endpoint descriptions) with all teammates (1)

Total for User Story 1.4: 12 story points

Team roles:

Christopher:	Vector Database {Product Owner, Developer}
Sai:	Frontend {Scrum Master, Developer}
Divya:	Prompting + QA {Developer}
Srushti:	Data Ingestion {Developer}
Ethan:	Backend + API {Developer}

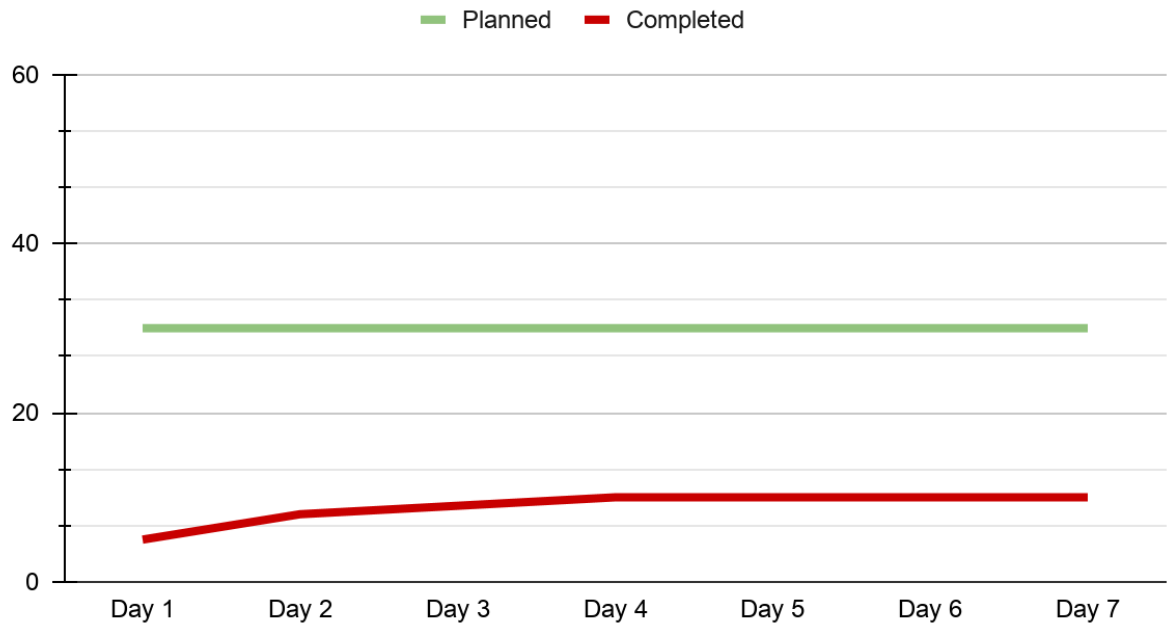
Initial task assignment:

Ethan:	User Story 1.4, Task 1
Christopher:	User Story 1.3, Task 3
Sai:	
Divya:	Spike 5
	User Story 1.2 Task 2
Srushti:	

Initial burnup chart: A graph giving the initial burnup chart for this sprint and is labeled as such with sprint number and project name and is located in the lab.

Sprint 1 Burnup Chart: Total planned work = 30 story points

Sprint 1 VeriFi Burnup Chart:



Initial scrum board: Also known as a task board, the scrum board is a physical board labeled as such with sprint number and project name and located in the lab. This board has four columns, titled user stories, tasks not started, tasks in progress, and tasks completed. Index cards or post-it notes representing the user stories and the tasks for this sprint should be placed in the user stories, tasks not started, and tasks in progress columns. Tasks associated with a user story should be placed in the same row as the user story.

User Stories	Tasks Not Started	Tasks In Progress	Tasks Completed
Infrastructure (5)			- Create Github repo - Write Readme - Verify Team Access
Spikes (12)	Research Embedding Models	- Research Vector Database - Research REST + FastAPI	- Research Prompt Engineering - Research RAG - Design Initial Prompt Template

Story 1.1 (5)		- User Interface Site Design - Submission button/textbox	
Story 1.2 (6)		Formatting parsed information into a .jsonl file / structure	Researched how retrieved document chunks, metadata, and citations formatted before being passed to the LLM
Story 1.3 (5)			- Completed small parser - Parser is able to generate .jsonl files ready for chunk scoring
Story 1.4 (8)	- Implement GET/HEALTH - Implement POST/CHAT - Connect Frontend to Backend	- Set-up dev env for FastAPI - Write API Contract	

Scrum times:

Tuesday (6/30) TA	Wednesday (7/1)	Friday (7/3) TA
9:30 AM	3:00 PM	10:00 AM